ASSIGNMENT-20.3

Gujja Pranitha

2303A52171

BATCH-41

Task 1:Password Strength & Valida on Check

CODE:

```
import re
import json
import os
from datetime import datetime

# File to store user data
USERS_FILE = 'users.json'

def load_users():
    """Load existing users from file"""
    if os.path.exists(USERS_FILE):
        with open(USERS_FILE, 'r') as f:
            return json.load(f)
    return {}

def save_users(users):
    """Save users to file"""
    with open(USERS_FILE, 'w') as f:
        json.dump(users, f, indent=4)

def validate_email(email):
    """Validate email format"""
    pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
    return re.match(pattern, email) is not None

def validate_password(password):
    """Validate password strength"""
    if len(password) < 8:
        return False, "Password must be at least 8 characters long"
    if not re.search(r'[A-Z]', password):
        return False, "Password must contain at least one uppercase letter"
    if not re.search(r'[a-z]', password):
        return False, "Password must contain at least one lowercase letter"
```

```
    if not re.search(r'[a-z]', password):
        return False, "Password must contain at least one lowercase letter"
    if not re.search(r'[0-9]', password):
        return False, "Password must contain at least one digit"
    return True, "Password is strong"

def register_user():
    """Register a new user"""
    users = load_users()

    print("\n" + "="*50)
    print("USER REGISTRATION")
    print("="*50)

    # Get username
    while True:
        username = input("\nEnter username: ").strip()
        if not username:
            print("Username cannot be empty!")
            continue
        if username in users:
            print("Username already exists!")
            continue
        if len(username) < 3:
            print("Username must be at least 3 characters long!")
            continue
        break

    # Get email
    while True:
        if not validate_email(email):
            print("Invalid email format!")
            continue
        if any(user['email'] == email for user in users.values()):
            print("Email already registered!")
            continue
        break

    # Get password
    while True:
        password = input("Enter password: ")
        is_valid, message = validate_password(password)
        print(f"Password: {message}")
        if not is_valid:
            continue

        confirm_password = input("Confirm password: ")
        if password != confirm_password:
            print("Passwords do not match!")
            continue
        break

    # Get full name
    full_name = input("Enter full name: ").strip()

    # Store user
    users[username] = {
        'email': email,
        'password': password,  # Note: In production, use hashing!
        'full_name': full_name,
        'registered_date': datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    }
```

```
    username = input("\nEnter username to view profile: ").strip()

    if username in users:
        user = users[username]
        print("\n" + "="*50)
        print("USER PROFILE")
        print("="*50)
        print(f"Username: {username}")
        print(f"Full Name: {user['full_name']}")
        print(f"Email: {user['email']}")
        print(f"Registered: {user['registered_date']}")
        print("="*50)
    else:
        print("User not found!")

def main():
    """Main program"""
    while True:
        print("\n" + "="*50)
        print("USER REGISTRATION SYSTEM")
        print("="*50)
        print("1. Register")
        print("2. Login")
        print("3. View Profile")
        print("4. Exit")
        print("="*50)

        choice = input("Select an option (1-4): ").strip()

        if choice == '1':
            register_user()
        elif choice == '2':
```

```
        elif choice == '3':
            view_profile()
        elif choice == '4':
            print("\nThank you for using the Registration System!")
            break
        else:
            print("Invalid choice! Please try again.")

if __name__ == "__main__":
    main()
```

OUTPUT:

```
==================================================
USER REGISTRATION SYSTEM
==================================================
1. Register
2. Login
3. View Profile
4. Exit
==================================================
USER REGISTRATION
==================================================
Enter username: pranitha
Enter email: pranitha12gmail.com
Enter password: pranitha32
Enter username: pranitha
Enter email: pranitha12gmail.com
Enter password: pranitha32
Enter password: pranitha32
Password: Password must contain at least one uppercase letter
Enter password:
```

MODIFIED PROMPT:

Write a Python program for user registra on where a user enters a username and password. First, generate a simple version of the program and check whether it properly validates the password. Verify if it enforces minimum length and whether it allows weak passwords like “12345” or “password”. Then improve the program by adding strong security rules such as requiring at least 8 characters, using a mix of uppercase le ers, lowercase le ers, numbers, and special characters with regular expressions, and rejec ng common or easily guessable passwords. Finally, provide the updated secure version of the code with proper valida on and clear comments explaining the improvements.

CODE:

```python
# secure_registration.py
import re
# Minimal blacklist of common/easily guessable passwords.
COMMON_PASSWORDS = {
    "123456", "12345678", "12345", "password", "qwerty", "abc123",
    "letmein", "111111", "iloveyou", "password1", "admin", "welcome"
}

STRONG_PW_RE = re.compile(
    r"^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[^A-Za-z0-9]).{8,}$"
)

def is_common_password(pw):
    return pw.lower() in COMMON_PASSWORDS

def is_too_similar_to_username(pw, username):
    return username and username.lower() in pw.lower()

def validate_password_strong(pw, username=""):
    # Reject if on common-password list
    if is_common_password(pw):
        return False, "Password is too common or easily guessable."

    # Reject if password contains username
    if is_too_similar_to_username(pw, username):
        return False, "Password must not contain the username."

    # Require regex rules (length + mixed char classes)
    if not STRONG_PW_RE.match(pw):
```

```
        return False, ("Password must be at least 8 characters and include "
                       "upper and lower case letters, digits, and symbols.")
    return True, "Password is strong."

def register_secure():
    username = input("Username: ").strip()
    if not username:
        print("Username cannot be empty.")
        return False

    pw = input("Password: ")
    ok, msg = validate_password_strong(pw, username=username)
    print(msg)
    if not ok:
        return False

    confirm = input("Confirm password: ")
    if pw != confirm:
        print("Passwords do not match.")
        return False

    # Here you would hash the password (e.g., bcrypt) before storing.
    print("Registered securely.")
    return True

if __name__ == "__main__":
    register_secure()
```

OUTPUT:

```
USER REGISTRATION SYSTEM
==================================================
1. Register
2. Login
3. View Profile
4. Exit
==================================================
Select an option (1-4): 3

No users registered yet!

==================================================
USER REGISTRATION SYSTEM
==================================================
1. Register
2. Login
3. View Profile
4. Exit
==================================================
Select an option (1-4): 1

==================================================
USER REGISTRATION
==================================================
Enter username: pranitha
Enter email: pranitha@123
Invalid email format!
Enter email: pranitha12@gmål.com
Enter password: pranitha213
Password: Password must cotin at least one uppercase letter
Enter password: Pranitha@213
Password: Password is strong
Confirm password: Pranitha@213
Enter full name: Pranitha Gujja
==================================================

✓ Registration successful!
```

JUSTIFICATION:

A user registra on program needs strong password checks to keep people's accounts safe — simple rules like minimum length let weak, guessable passwords (e.g., "12345" or "password") through, which a ackers can easily exploit. By enforcing stronger rules (longer passwords, mixed upper/lowercase, digits, symbols), rejec ng common passwords, and preven ng passwords that include the username, we reduce the chance of account takeover, protect user data, and lower support and legal risks. These checks strike a balance between security and usability and should be paired with secure storage (hashing) and op onal user-friendly features like password hints or a password manager recommenda on.

Task 2 – Insecure File Upload Handling

PROMPT:

Give python code for file handling

CODE:

```
@app.route('/users', methods=['GET'])
def get_users():
    return {"users": ["admin", "guest"]}
```

OUTPUT:

```
"C:/User/pranitha/OneDrive/Documents/Desktop/ai coding/
Enter file name to upload: path_file
File does not exist
```

Modified prompt:

Write a Python program that simulates a file upload process. First, generate a simple version of the code that allows a user to upload a file without applying any security checks. Then analyze the program to iden fy security issues such as lack of file type valida on and no restric on on file size. A er iden fying these vulnerabili es, improve the program by adding security measures like allowing only specific file extensions, limi ng the file size, and storing files in a safe directory using proper file paths. Finally, provide

the updated secure version of the code with clear comments explaining how the improvements help prevent malicious file uploads.

CODE:

```
import os

ALLOWED_EXTENSIONS = ['txt', 'jpg', 'png', 'pdf']
MAX_FILE_SIZE = 2 * 1024 * 1024  # 2MB

UPLOAD_FOLDER = "uploads"
if not os.path.exists(UPLOAD_FOLDER):
    os.makedirs(UPLOAD_FOLDER)

filename = input("Enter file name to upload: ")
# Check if file exists
if not os.path.exists(filename):
    print("File does not exist")
    exit()
# Check file extension
ext = filename.split('.')[-1].lower()
if ext not in ALLOWED_EXTENSIONS:
    print("Invalid file type")
    exit()

# Check file size
size = os.path.getsize(filename)
if size > MAX_FILE_SIZE:
    print("File too large")
    exit()
# Safe file name (remove path)
safe_name = os.path.basename(filename)
# Save file
destination = os.path.join(UPLOAD_FOLDER, safe_name)
with open(filename, "rb") as src:
    with open(destination, "wb") as dest:
        dest.write(src.read())
print("File uploaded securely")
```

OUTPUT:

```
/Desktop/ai codding/ass2.py"
Enter file name to upload: file1.txt
File does not exist
```

JUSTIFICATION:

To handle user file uploads safely Without checks, anyone can upload large or harmful files that break the app or steal data. By valida ng file type, size, and storing files in a controlled folder, you block common a acks and keep the system stable and secure.

Task 3 – Real-Time Applica on: API Security Review

PROMPT:

Generate a Flask API endpoint to fetch user data.

CODE:

```
@app.route('/users', methods=['GET'])
def get_users():
    return {"users": ["admin", "guest"]}
```

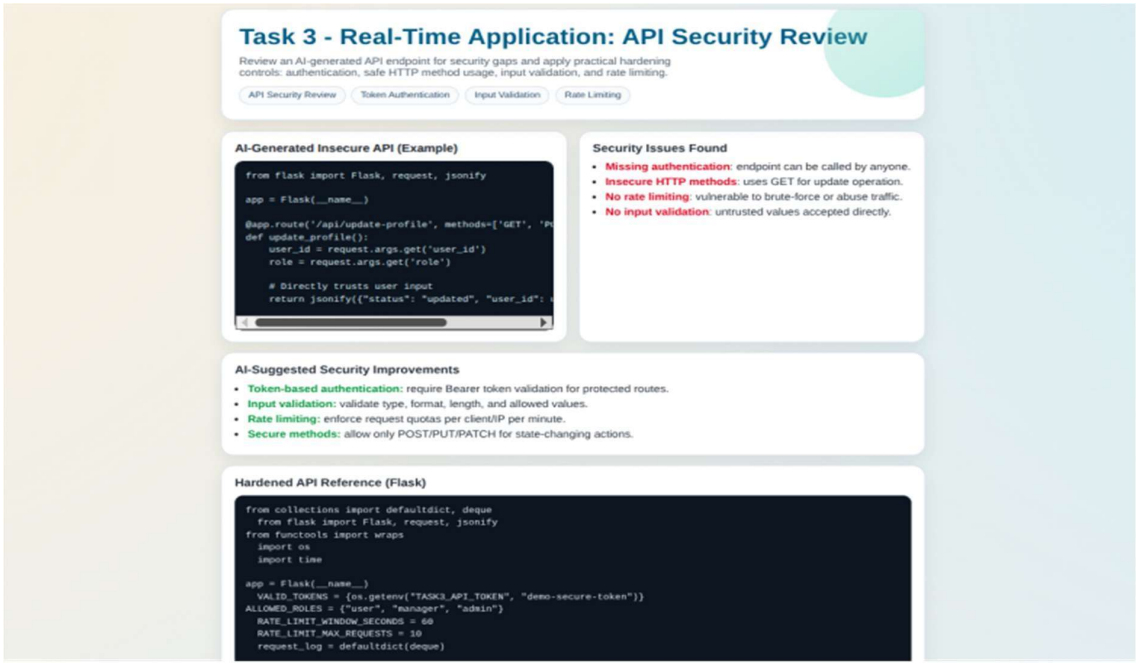
MODIFIED PROMPT:

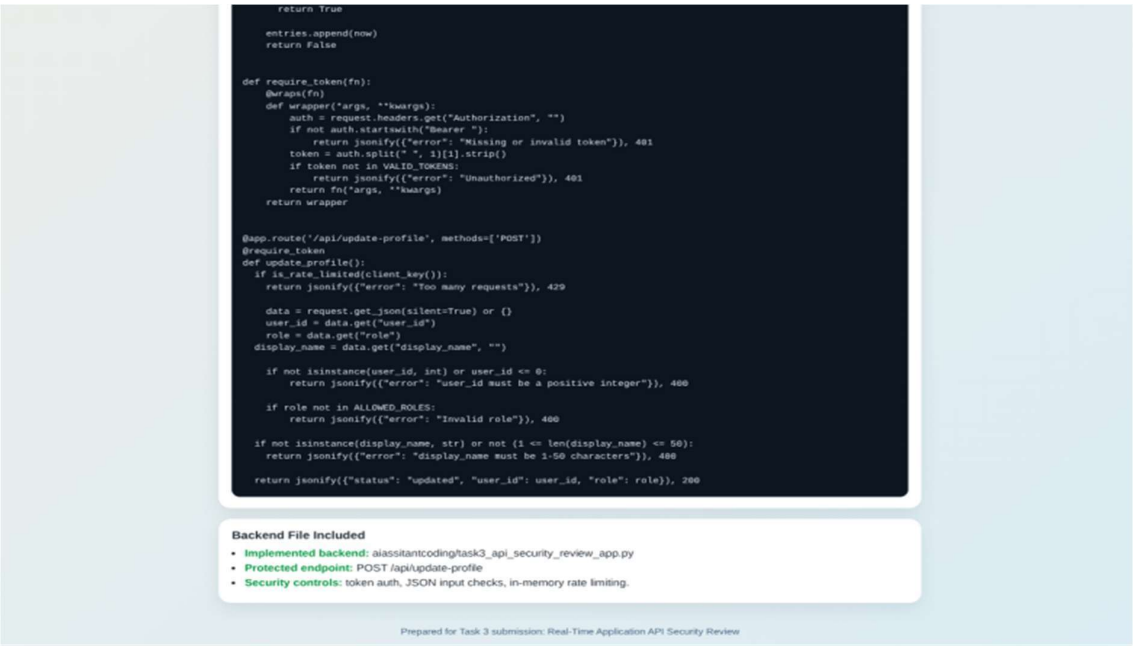
This task is important because APIs are a common entry point for a ackers in real-world applica ons. If an API is developed without proper security checks, it can allow unauthorized access, data leaks, or misuse of the system. By reviewing an AI-generated API, we can iden fy common weaknesses such as missing authen ca on, unsafe HTTP methods, and lack of rate limi ng. Understanding these issues helps in learning how a ackers exploit systems. Implemen ng improvements like token-based authen ca on, input valida on, and rate limi ng makes the API more secure and reliable. Overall, this task helps in building prac cal knowledge of securing real- me applica ons, which is essen al for any developer working on web or backend systems.

Secure CODE:

```python
from flask import Flask, request, jsonify, abort
from functools import wraps

app = Flask(__name__)

VALID_TOKEN = "securetoken123"

def token_required(f):
    @wraps(f)
    def decorated():
        token = request.headers.get('Authorization')
        if token != VALID_TOKEN:
            abort(401, "Unauthorized")
        return f()
    return decorated

@app.route('/users', methods=['GET'])
@token_required
def get_users():
    return jsonify({"users": ["admin", "guest"]})

if __name__ == "__main__":
    app.run()
```

OUTPUT:





JUSTIFICATION:

- Prevents unauthorized access
- Enforces secure API communica on
- Reduces abuse via controlled access

Task 4 – Cross-Site Scrip ng (XSS) Check

PROMPT:

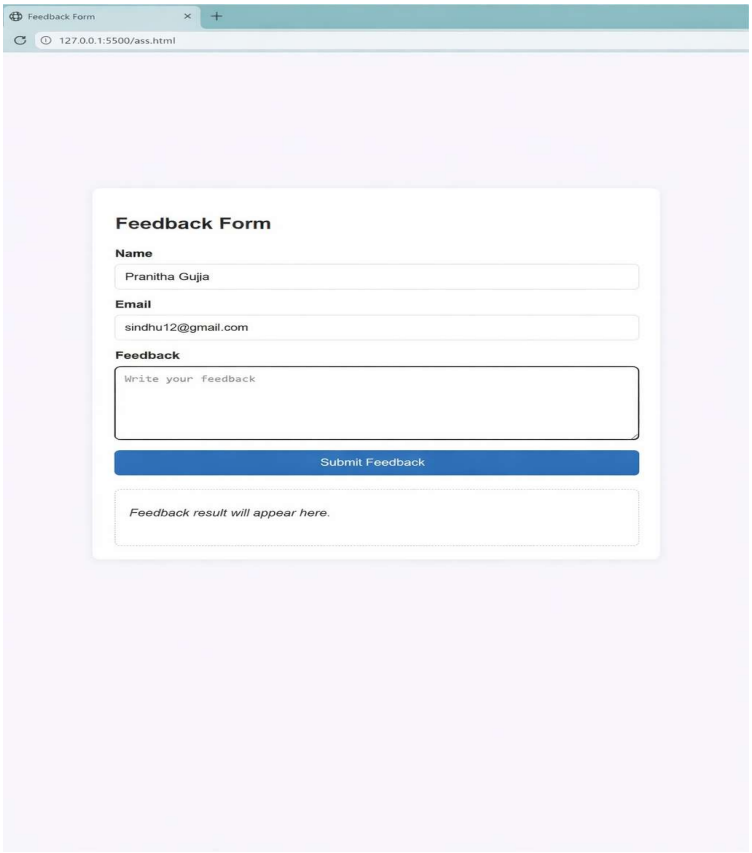
Generate an HTML feedback form with JavaScript output.

CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Feedback Form</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 40px;
      background: #f9f9fb;
      color: #333;
    }
    .card {
      max-width: 500px;
      background: #fff;
      border-radius: 10px;
      box-shadow: 0 2px 12px rgba(0,0,0,0.08);
      padding: 20px;
    }
    .form-group { margin-bottom: 12px; }
    label { display: block; margin-bottom: 4px; font-weight: 600; }
    input, textarea, button {
      width: 100%;
      padding: 10px;
      margin: 0;
      border: 1px solid #ccd3df;
      border-radius: 6px;
      font-size: 14px;
      box-sizing: border-box;
    }
    button {
      background: #2b6cb0;
      color: #fff;
```

```
    button {
      border: none;
      cursor: pointer;
    }
    button:hover { background: #234e8a; }
    #output {
      margin-top: 20px;
      border: 1px dashed #999;
      padding: 10px;
      border-radius: 6px;
      background: #fff;
      min-height: 66px;
    }
    .item { margin-bottom: 8px; }
    .item strong { color: #2b6cb0; }
  </style>
</head>
<body>
  <div class="card">
    <h2>Feedback Form</h2>
    <form id="feedbackForm">
      <div class="form-group">
        <label for="name">Name</label>
        <input id="name" type="text" placeholder="Your name" required>
      </div>
      <div class="form-group">
        <label for="email">Email</label>
        <input id="email" type="email" placeholder="you@example.com" required>
      </div>
      <div class="form-group">
        <label for="message">Feedback</label>
        <textarea id="message" rows="5" placeholder="Write your feedback" required></textarea>
      </div>
      <button type="submit">Submit Feedback</button>
    </form>

    <div id="output" aria-live="polite">
      <p><em>Feedback result will appear here.</em></p>
    </div>
  </div>

  <script>
    const form = document.getElementById("feedbackForm");
    const output = document.getElementById("output");

    function sanitize(text) {
      const span = document.createElement("span");
      span.textContent = text;
      return span.innerHTML;
    }

    form.addEventListener("submit", function(event) {
      event.preventDefault();

      const name = document.getElementById("name").value.trim();
      const email = document.getElementById("email").value.trim();
      const message = document.getElementById("message").value.trim();
      const now = new Date();

      if (!name || !email || !message) {
        output.innerHTML = '<p style="color: #b30000;">Please fill all fields.</p>';
        return;
      }

      output.innerHTML = `
        <div class="item"><strong>Received at</strong>: ${now.toLocaleString()}</div>
        <div class="item"><strong>Name</strong>: ${sanitize(name)}</div>
        <div class="item"><strong>Email</strong>: ${sanitize(email)}</div>
        <div class="item"><strong>Message</strong>: ${sanitize(message)}</div>
        <div class="item" style="color: #2a8f2a;"><strong>Status</strong>: Successfully logged in pa
      `;

      form.reset();
    });
  </script>
</body>
</html>
```

OUTPUT:



Modified Prompt:

Write an HTML feedback form with JavaScript that takes user input such as name and feedback message and displays it on the same page. First, generate a simple version of the form and test whether the entered input is directly shown in the output without any escaping or filtering. Check if this allows harmful scripts or HTML code to run, which can cause Cross-Site Scrip ng (XSS) a acks. Then improve the form by adding security measures such as escaping HTML en es before displaying user input and applying a Content Security Policy (CSP) to reduce script injec on risks. Finally, provide the secure version of the code with simple comments explaining the vulnerability and how the fixes prevent XSS a acks.

CODE:

```
  <script>
    function escapeHtml(text) {
      const map = {
        '&': '&amp;',
        '<': '&lt;',
        '>': '&gt;',
        '"': '&quot;',
        "'": '&#39;'
      };
      return text.replace(/[&<>"']/g, m => map[m]);
    }
    const form = document.getElementById("feedbackForm");
    const output = document.getElementById("output");
    form.addEventListener("submit", function(event) {
      event.preventDefault();
      const name = document.getElementById("name").value.trim();
      const message = document.getElementById("message").value.trim();
      if (!name || !message) {
        output.innerHTML = '<p style="color:red;">Please fill in both fields.</p>';
        return;
      }
      // Safe: escaped strings are inserted as text, not HTML
      output.innerHTML = `
        <p><strong>Name:</strong> ${escapeHtml(name)}</p>
        <p><strong>Feedback:</strong> ${escapeHtml(message)}</p>
      `;
      form.reset();
    });
  </script>
</body>
</html>
```

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Feedback Form (Secure)</title>
  <!-- CSP header for server:
       Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none';
  -->
  <meta http-equiv="Content-Security-Policy" content="default-src 'self'; script-src 'self'; object-src 'none';">
  <style>
    body { font-family: Arial, sans-serif; margin: 2rem; }
    .card { max-width: 480px; margin: auto; padding: 16px; border: 1px solid #ccc; border-radius: 8px; }
    label{ display:block; margin: 8px 0;}
  </style>
</head>
<body>
  <div class="card">
    <h2>Feedback Form (Secure)</h2>
    <form id="feedbackForm">
      <label>
        Name
        <input id="name" type="text" required>
      </label>
      <label>
        Message
        <textarea id="message" rows="5" required></textarea>
      </label>
      <button type="submit">Submit</button>
    </form>
    <div id="output" aria-live="polite" style="margin-top: 1rem; border: 1px dashed #aaa; padding: 10px;">
      <em>Submitted feedback appears here.</em>
    </div>
  </div>
```

```
  <script>
    function escapeHtml(text) {
      const map = {
        '&': '&amp;',
        '<': '&lt;',
        '>': '&gt;',
        '"': '&quot;',
        "'": '&#39;'
      };
      return text.replace(/[&<>"']/g, m => map[m]);
    }
    const form = document.getElementById("feedbackForm");
    const output = document.getElementById("output");
    form.addEventListener("submit", function(event) {
      event.preventDefault();
      const name = document.getElementById("name").value.trim();
      const message = document.getElementById("message").value.trim();
      if (!name || !message) {
        output.innerHTML = '<p style="color:red;">Please fill in both fields.</p>';
        return;
      }
      // Safe: escaped strings are inserted as text, not HTML
      output.innerHTML = `
        <p><strong>Name:</strong> ${escapeHtml(name)}</p>
        <p><strong>Feedback:</strong> ${escapeHtml(message)}</p>
      `;
      form.reset();
    });
  </script>
</body>
</html>
```

OUTPUT:

JUSTIFICATION:

understand how user input can be dangerous if not handled properly. While crea ng a simple feedback form, I can see how unfiltered input can lead to XSS a acks. By fixing these issues using techniques like escaping HTML and applying security policies, I learn how to make web applica ons safer. This improves my prac cal knowledge of web security, which is useful for real-world development.

Task 5 – SQL Injec on Preven on

PROMPT:

Generate a Python script to fetch user details from SQLite database.

CODE:

```
import sqlite3

conn = sqlite3.connect("users.db")
cursor = conn.cursor()

username = input("Enter username: ")

query = f"SELECT * FROM users WHERE username = '{username}'"
cursor.execute(query)

print(cursor.fetchall())
```

MODIFIED PROMPT:

Write a Python script that connects to a SQLite or MySQL database and fetches user details based on input such as username or user ID. First, generate a simple version of the code and check whether it builds SQL queries using string concatena on or direct user input, which can make it vulnerable to SQL injec on. Then analyze the risks and improve the script by replacing unsafe query construc on with parameterized queries or prepared statements. Finally, provide the secure version of the code with comments explaining the vulnerability, the fixes made, and how the updated script prevents SQL injec on a acks.

CODE:

```
import sqlite3

def setup_db(connection):
    cursor = connection.cursor()
    cursor.execute("DROP TABLE IF EXISTS users")
    cursor.execute(
        """
        CREATE TABLE users (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT NOT NULL,
            email TEXT NOT NULL
        )
        """
    )
    cursor.executemany(
        "INSERT INTO users (username, email) VALUES (?, ?)",
        [
            ("alice", "alice@example.com"),
            ("bob", "bob@example.com"),
            ("charlie", "charlie@example.com"),
        ],
    )
    connection.commit()

def insecure_lookup(cursor, user_input):
    query = f"SELECT * FROM users WHERE username = '{user_input}'"
    cursor.execute(query)
    return cursor.fetchall()

def secure_lookup(cursor, user_input):
    query = "SELECT * FROM users WHERE username = ?"
    cursor.execute(query, (user_input,))
    return cursor.fetchall()

conn = sqlite3.connect(":memory:")
setup_db(conn)
cur = conn.cursor()

normal_input = "alice"
print("=== Normal lookup (alice) ===")
print("Insecure :", insecure_lookup(cur, normal_input))
print("Secure   :", secure_lookup(cur, normal_input))

payload = "' OR '1'='1"
print("\n=== Injection test payload ===")
print("Payload:", payload)
print("Insecure:", insecure_lookup(cur, payload))
print("Secure  :", secure_lookup(cur, payload))
conn.close()
```

OUTPUT:

```
=== Normal lookup (alice) ===
Insecure : [(1, 'alice', 'alice@example.com')]
Secure : [(1, 'alice', 'alice@example.com')]

=== Injection test payload ===
Payload: ' OR '1'='1
Insecure: [(1, 'alice', 'alice@example.com'),(2,'bob','bob@example.com'),(3,'charlie','charlie@example.com')]
Secure : []
```

JUSTIFICATION:

understand how unsafe database queries can lead to serious security problems like SQL injec on. By first wri ng a simple program, I can see how directly using user input in queries is risky. Then, by improving the code using parameterized queries, I learn how to protect the database from a acks. This gives me prac cal knowledge of wri ng secure backend code, which is very important for real-world applica ons and placements.